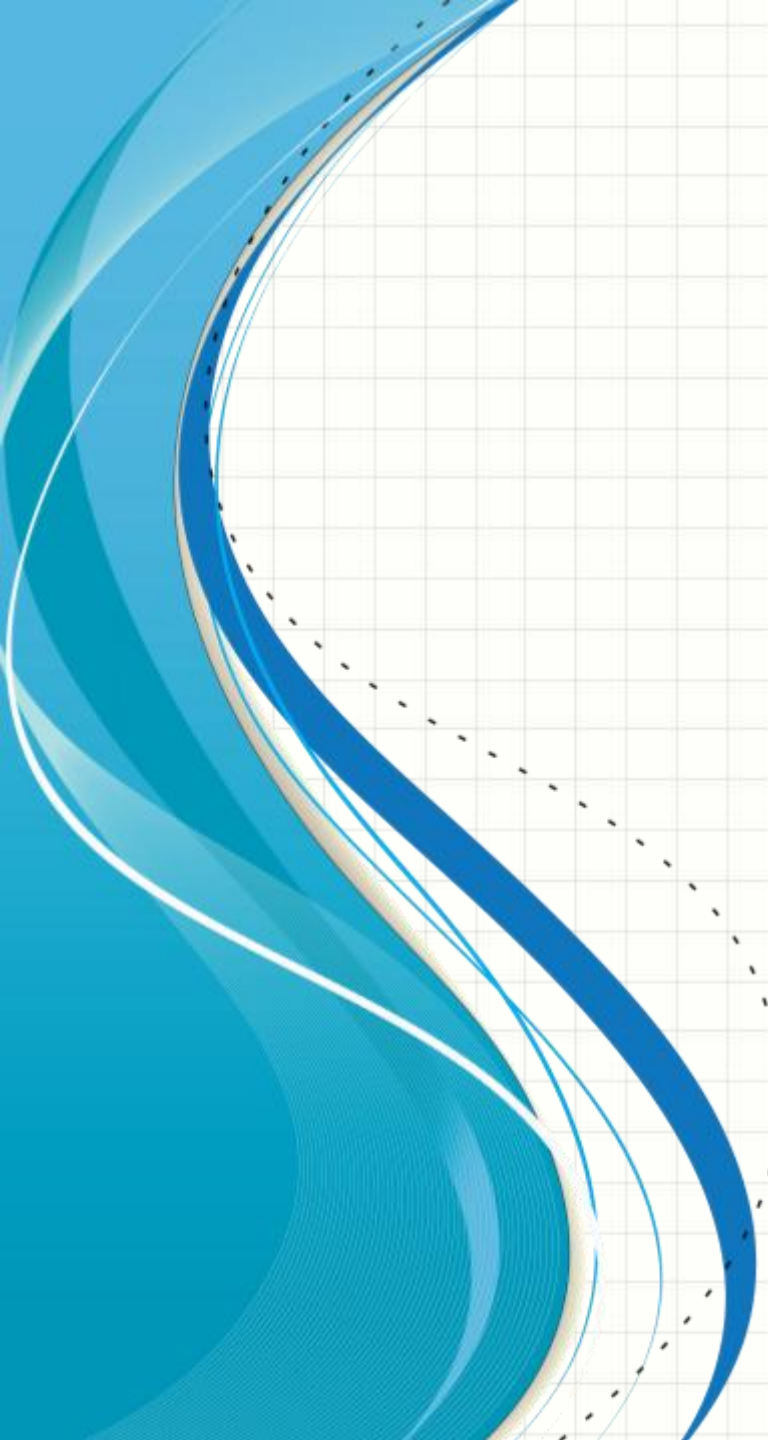




CHAPTER # 4

COMBINATIONAL LOGIC

Lecturer: Noman Al Hassan
Email: noman44m@hotmail.com

Objective of Lecture

After studying this chapter, you should be able to know

- Introduction.
 - Combinational Logic Circuits & Sequential Logic Circuits.
- Combinational Logic Circuits.
- Design Procedure.
- Binary Adder, subtractor.
 - *Half Adder / Subtractor*
 - *Full Adder / Subtractor*
 - Binary parallel Adder / *Subtractor*.
- Decoders.
- Encoders.
- Multiplexers.
- Demultiplexer.

Introduction

- Logic circuits for digital systems may be....

Combinational circuits

- Consists of logic gates whose output depends on only the present input.

- Example: Adder

$$\begin{array}{r} 1 \\ 0 \\ + \hline 1 \end{array}$$

Sequential circuits

- Consists of logic gates + storage elements. Output depends on inputs and the state of the storage elements or previous outputs.

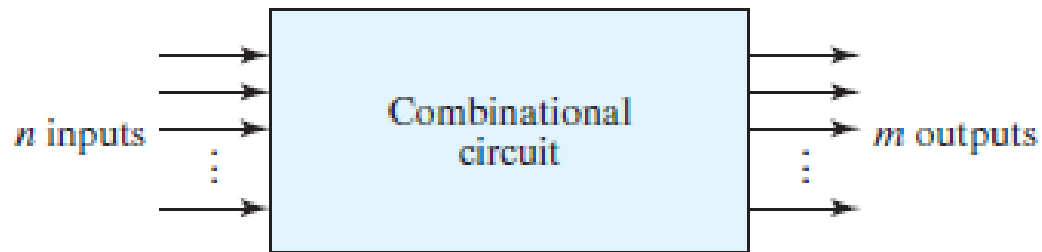
- Example: Counter

- $4+1 \rightarrow 5$
- $3+1 \rightarrow 4$

Sequential circuits are the building blocks of digital systems and will be discussed in next chapter.

Combinational Logic Circuits

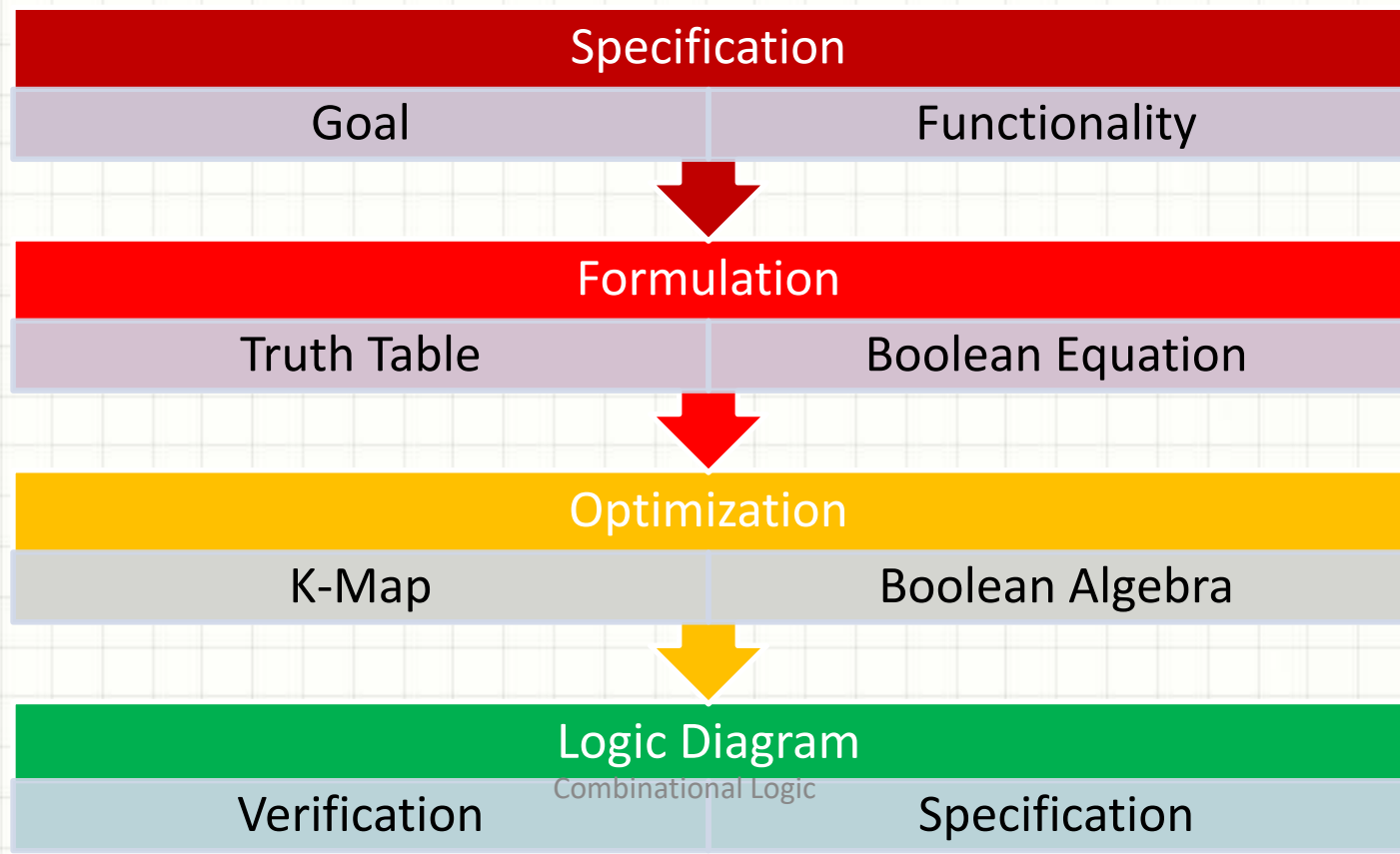
- Consists of logic gates .
- React to the values of the input signals and produce the output signals.



- Chapter 1 , we learned about binary numbers and variables.
- In Chapter 2 , we introduced Boolean algebra.
- In Chapter 3 , we learned how to simplify Boolean functions.
- The purpose of the current chapter is to use the knowledge acquired in previous chapters to formulate systematic analysis and design procedures for combinational circuits.

Design Procedure

- Starts from the specification of the problem, which leads to the truth table. Using the output values of truth table, the logic equation and simplified using K maps, or Algebraic manipulation. The equation of the output functions, the corresponding circuit is found. The process is shown in Figure:



Example 1

- Design a circuit that has a 3-bit input and a single output (F) specified as follows:
 - $F = 0$, when the input is less than (5)
 - $F = 1$, otherwise

Solution:

- **Step 1 (Specification):**
- Label the inputs (3 bits) as X, Y, Z
 - X is the most significant bit, Z is the least significant bit
- The output (1 bit) is F:
 - $F = 1 \rightarrow (101)_2, (110)_2, (111)_2$
 - $F = 0 \rightarrow$ other inputs

Example 1 (cont.)

Step 2 (Formulation)

Obtain Truth table

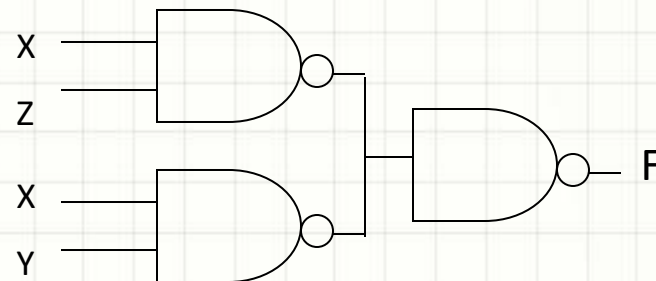
X	Y	Z	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Step 3 (Optimization)

X \ YZ					
	00	01	11	10	
0	0	0	0	0	$F = XZ + XY$
1	0	1	1	1	

Step 4 (Logic Diagram)

Circuit Diagram



Design Example 2:

BCD to Excess-3 Converter

Step 1 (Specification)

- 4-bit BCD input (A,B,C,D)
- 4-bit E-3 output (W,X,Y,Z)

Step 2 (Formulation)

Obtain Truth table

Decimal	BCD Input				Excess 3 Output			
	A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	1	1
1	0	0	0	1	0	1	0	0
2	0	0	1	0	0	1	0	1
3	0	0	1	1	0	1	1	0
4	0	1	0	0	0	1	1	1
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0
10-15	All other inputs				X	X	X	X

Example 2 (cont.)

Step 3 (Optimization)

AB \ CD	00	01	11	10
00	0	0	0	0
01	0	1	1	1
11	X	X	X	X
10	1	1	X	X

$$W = A + BC + BD$$

AB \ CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	X	X	X	X
10	0	1	X	X

$$X = B'C + BC'D' + B'D$$

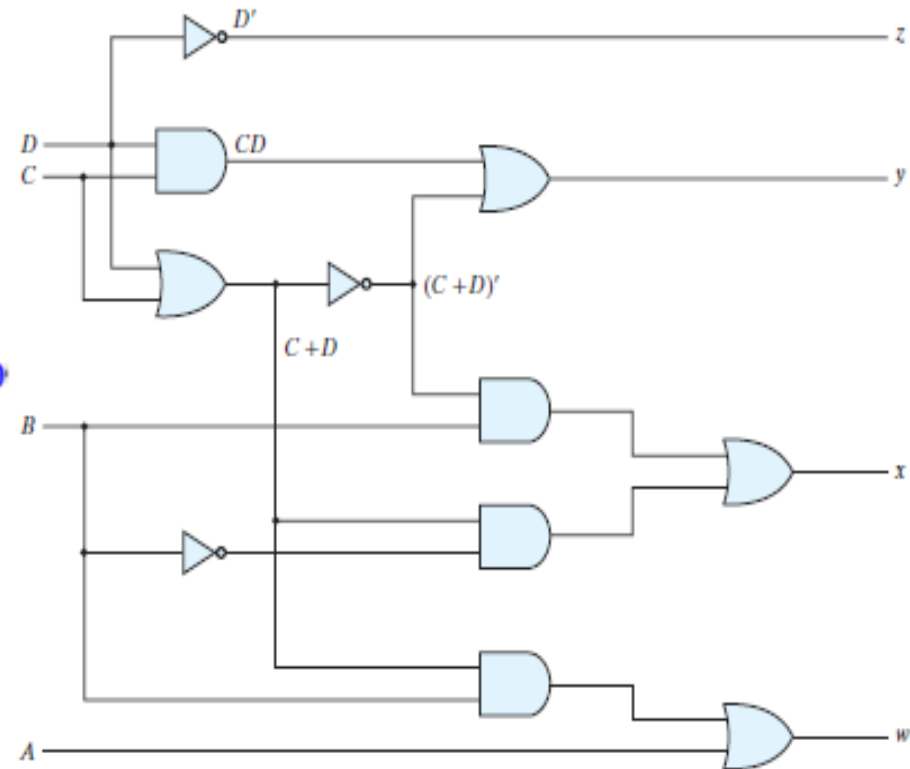
AB \ CD	00	01	11	10
00	1	0	1	0
01	1	0	1	0
11	X	X	X	X
10	1	0	X	X

$$Y = CD + C'D'$$

AB \ CD	00	01	11	10
00	1	0	0	1
01	1	0	0	1
11	X	X	X	X
10	1	0	X	X

$$Z = D'$$

Step 4 (Logic Diagram)



Logic diagram for BCD-to-excess-3 code converter

BINARY ADDER–SUBTRACTOR

$$0 + 0 = 0$$

$$0 + 1 = 1$$

$$1 + 0 = 1$$

$$1 + 1 = 10$$

- A combinational circuit that performs the addition of two bits is called a half adder .
- One that performs the addition of three bits is a full adder .
- half adders + half adders = full adder.

Half Adder

Step 1 (Specification)



Step 2 (Formulation)

Half Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Step 3 (Optimization)

For Carry

	$\bar{x}$	x
y	0	1
0	0	0
1	0	1

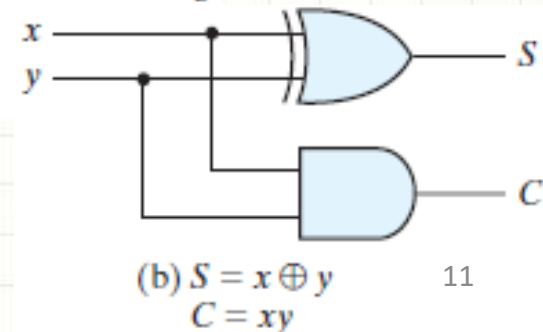
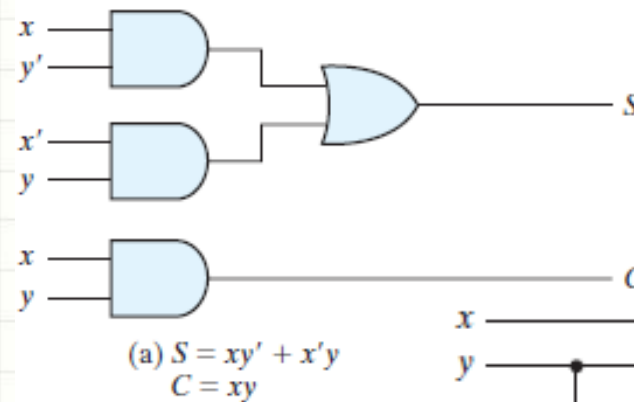
$$\text{Carry} = XY$$

For Sum

	$\bar{x}$	x
y	0	1
0	0	1
1	1	0

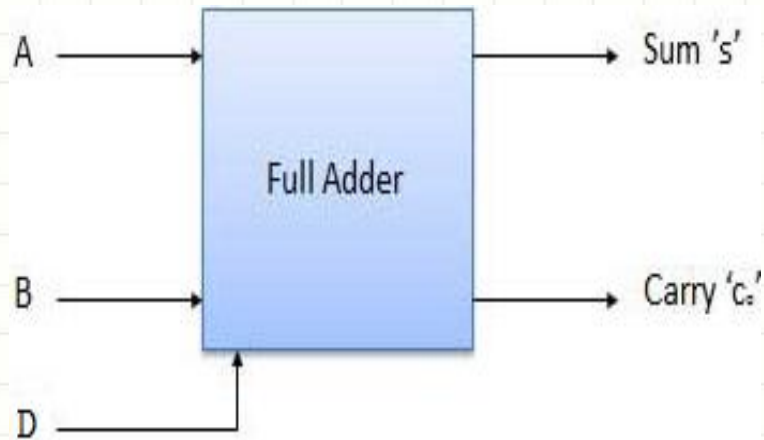
$$\begin{aligned} \text{Sum} &= XY' + X'Y \\ &= X \oplus Y \end{aligned}$$

Step 4 (Logic Diagram)



Full Adder

Step 1 (Specification)



Step 2 (Formulation)

Full Adder

<i>x</i>	<i>y</i>	<i>z</i>	<i>C</i>	<i>S</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Full Adder

Step 3 (Optimization)

yz		y			
		00	01	11	10
x	0	m_0	m_1 1	m_3	m_2 1
	1	m_4 1	m_5	m_7 1	m_6

z

(a) $S = x'y'z + x'yz' + xy'z' + xyz$

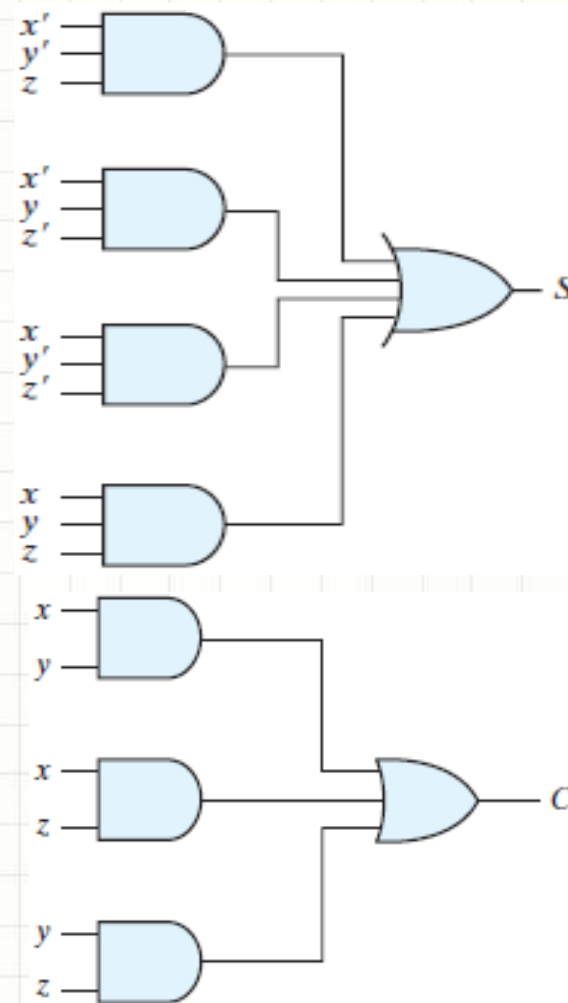
yz		y			
		00	01	11	10
x	0	m_0	m_1	m_3 1	m_2
	1	m_4	m_5 1	m_7 1	m_6 1

z

(b) $C = xy + xz + yz$

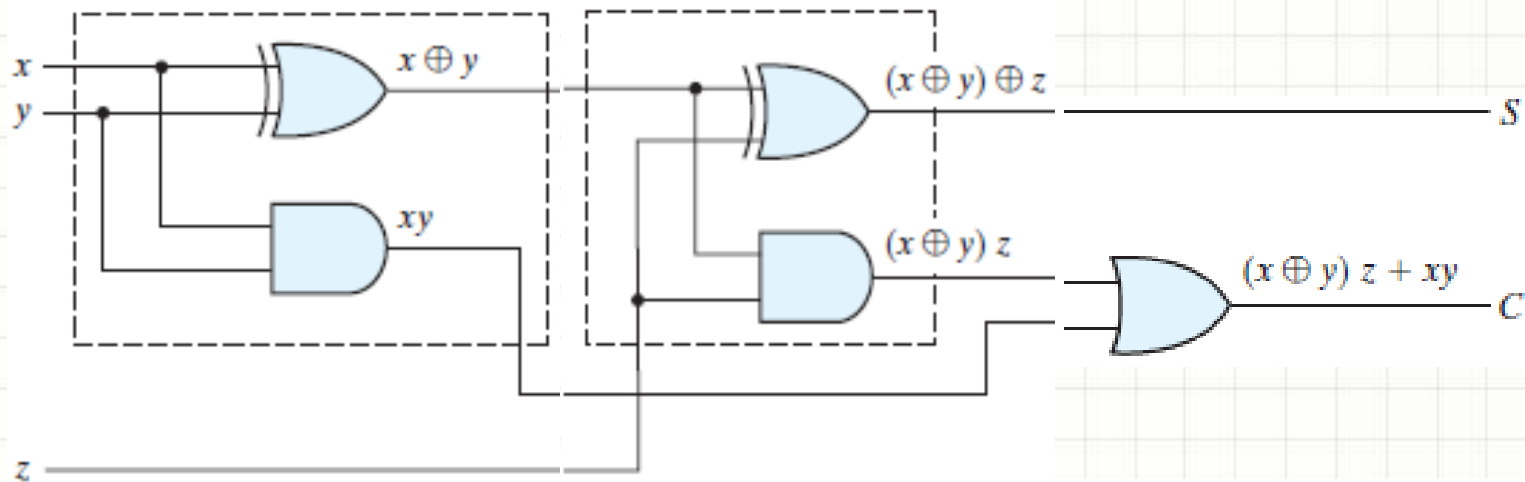
K-Maps for full adder

Step 4 (Logic Diagram)



Implementation of full adder in sum-of-products form

Full Adder = Two Half Adder



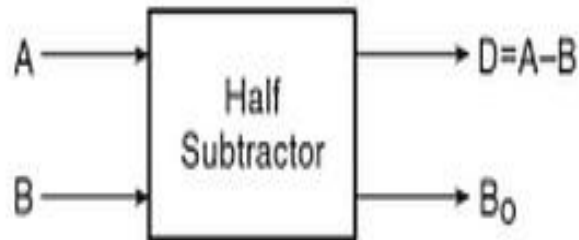
Implementation of full adder with two half adders and an OR gate

$$\begin{aligned} S &= z \oplus (x \oplus y) \\ &= z'(xy' + x'y) + z(xy' + x'y)' \\ &= z'(xy' + x'y) + z(xy + x'y') \\ &= xy'z' + x'yz' + xyz + x'y'z \end{aligned}$$

$$C = z(xy' + x'y) + xy = xy'z + x'yz + xy$$

Half Subtractor

Step 1 (Specification)



Step 2 (Formulation)

Inputs		Outputs	
A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Table 3.8 Truth table for half-subtractor

Step 3 (Optimization)

For Difference

A \ B	0	1
0	0	1
1	1	0

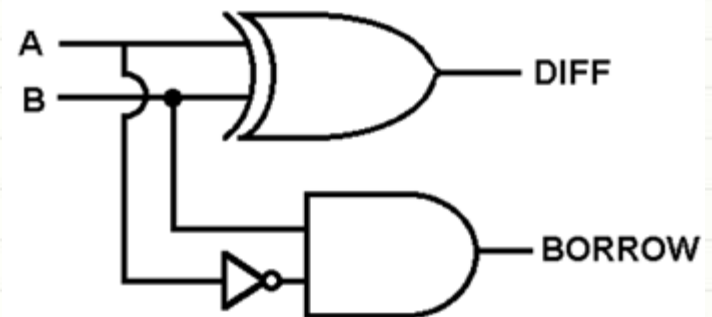
$$\begin{aligned}\text{Difference} &= A\bar{B} + \bar{A}B \\ &= A \oplus B\end{aligned}$$

For Borrow

A \ B	0	1
0	0	1
1	0	0

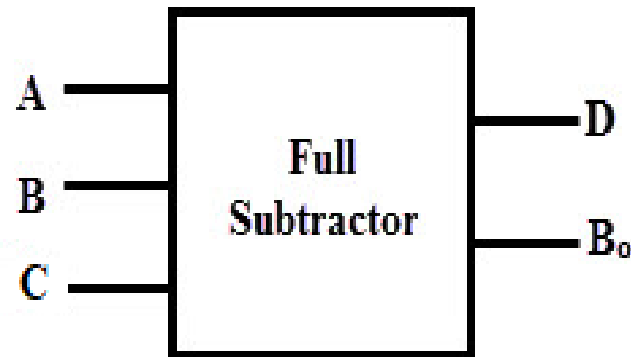
$$\text{Borrow} = \bar{A}B$$

Step 4 (Logic Diagram)



Full Subtractor

Step 1 (Specification)



Block Diagram of
Full Subtractor

Step 2 (Formulation)

Full Subtractor Truth Table:

Inputs			Outputs	
X	Y	Z	D	B-out
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$S(X, Y, Z) = \Sigma (1, 2, 4, 7)$$

$$B\text{-out}(x, y, Z) = \Sigma (1, 2, 3, 7)$$

$$X - Y - Z$$

Full Subtractor

Step 3 (Optimization)

Difference D

		X			
	XY	00	01	11	10
B-in	0	0	2 1	6	4 1
1	1	1	3	7 1	5

Y

$$D = X'Y'(B-in) + X'Y(B-in)' + XY'(B-in)' + XY(B-in)$$

$$D = X \oplus Y \oplus (B-in)$$

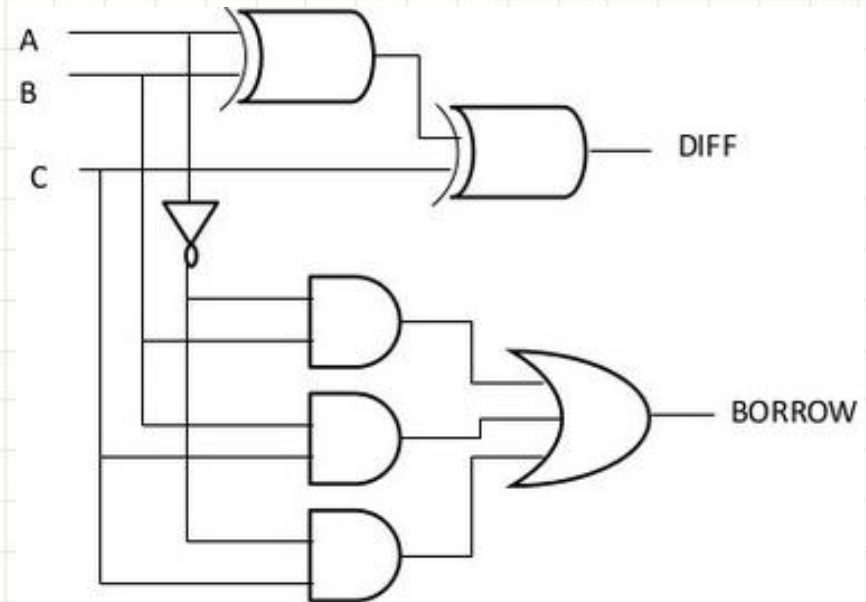
Borrow B-out

		X			
	XY	00	01	11	10
B-in	0	0	1	6	4
1	1	1	1	7 1	5

Y

$$B-out = X'Y + X'(B-in) + Y(B-in)$$

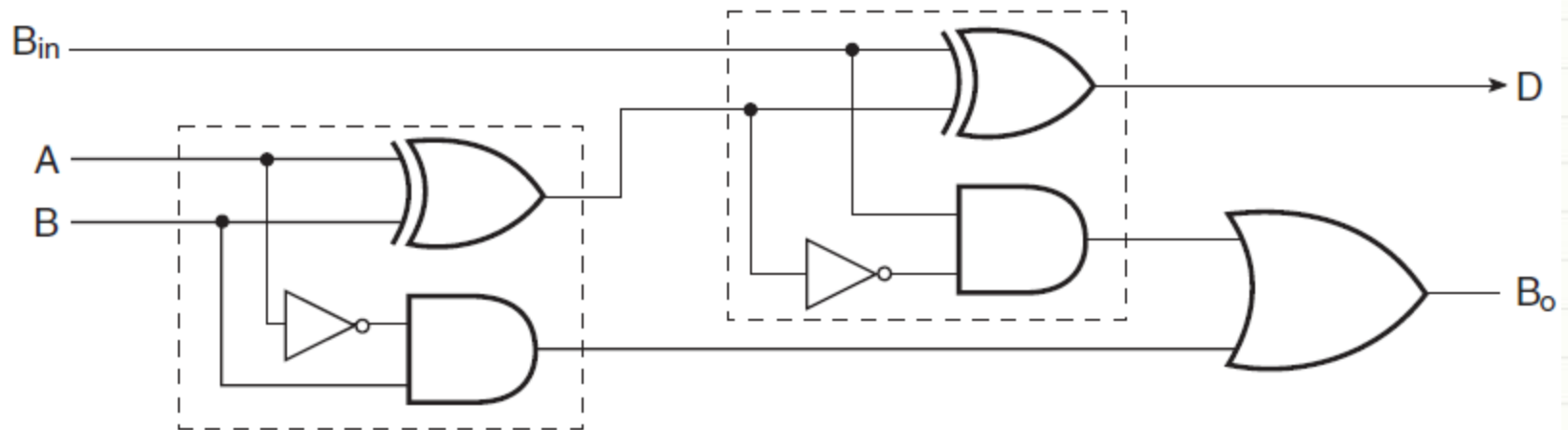
Step 4 (Logic Diagram)



$$D = X \oplus Y \oplus (B-in)$$

$$B-out = X'Y + X'(B-in) + Y(B-in)$$

Full Subtractor = Two Half Subtractor



$$D = X \oplus Y \oplus (B-in)$$

$$B-out = X'Y + X'(B-in) + Y(B-in)$$

Implementation of full-subtractor with two half subtractor and an OR gate

Decoders

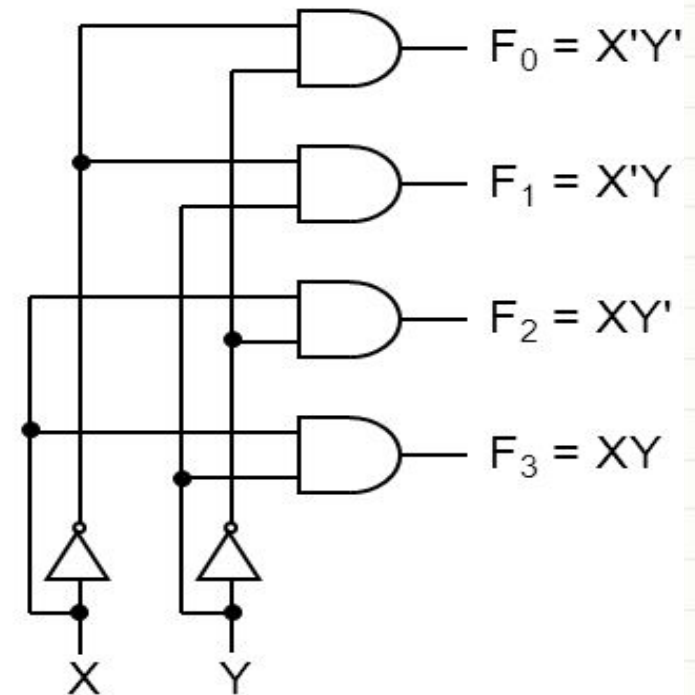
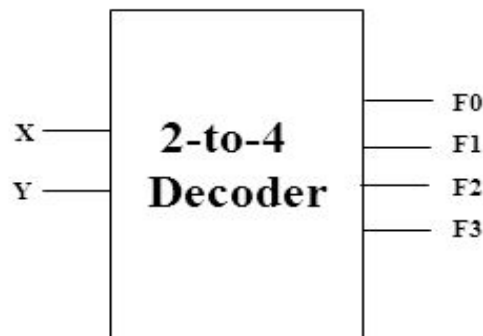
A device that converts signals from one form to another

2-to-4 Binary Decoder

Truth Table:

X	Y	F_0	F_1	F_2	F_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

- From truth table, circuit for 2x4 decoder is:
- Note: Each output is a 2-variable minterm ($X'Y'$, $X'Y$, XY' or XY)



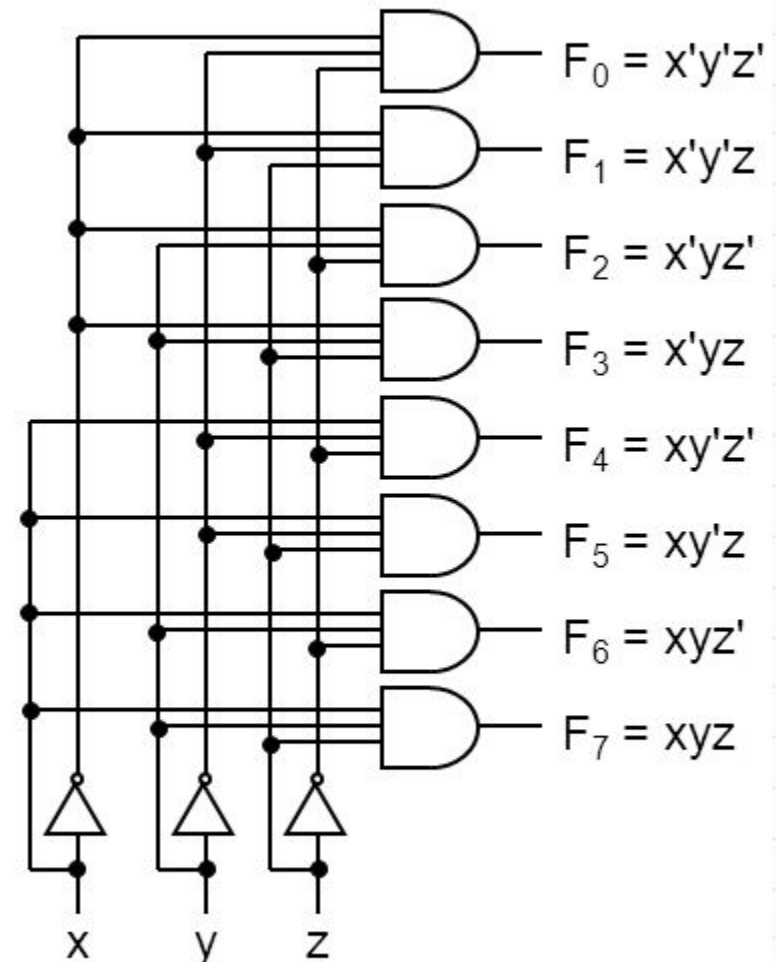
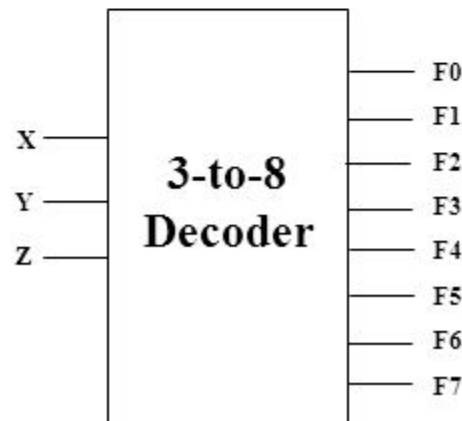
- N Inputs = 2^N Outputs
 - 2 to 4, 3 to 8, 4 to 16

Decoders

3-to-8 Binary Decoder

Truth Table:

x	y	z	F ₀	F ₁	F ₂	F ₃	F ₄	F ₅	F ₆	F ₇
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

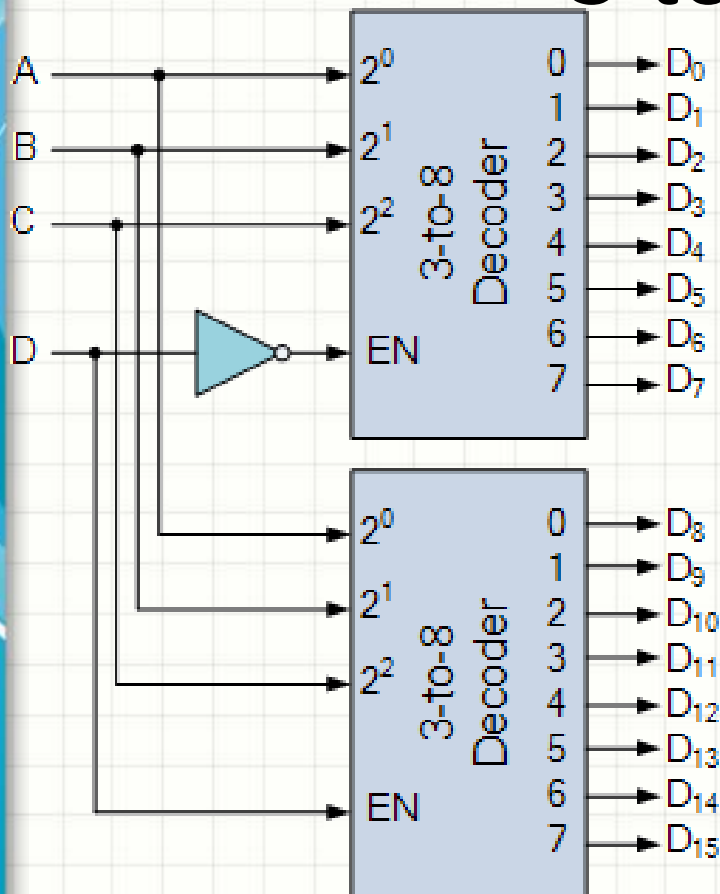


- N Inputs = 2^N Outputs

Combinational Logic

- 2 to 4, 3 to 8, 4 to 16

4 to 16 Decoder implemented with 3 to 8 decoder

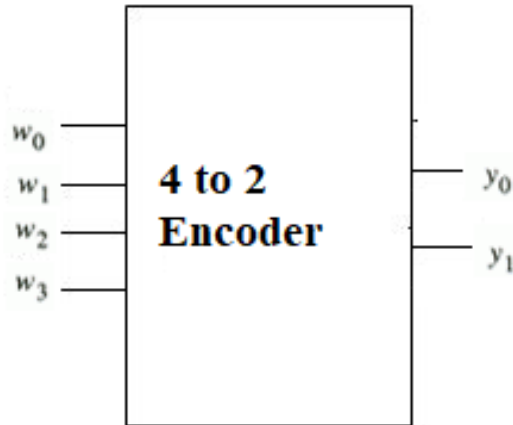


W	X	Y	Z	M0	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10	M11	M12	M13	M14	M15
0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	1	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
0	1	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0
0	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
0	1	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
1	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0
1	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0
1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

4-to-16 Line Decoder Implemented
with two 3-to-8 Decoders

Encoders (4 to 2)

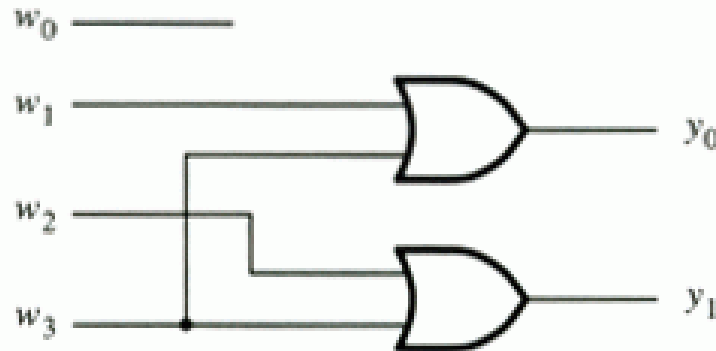
A device that converts information from one format or code to another.



Block Diagram

w_0	w_1	w_2	w_3	y_1	y_0
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

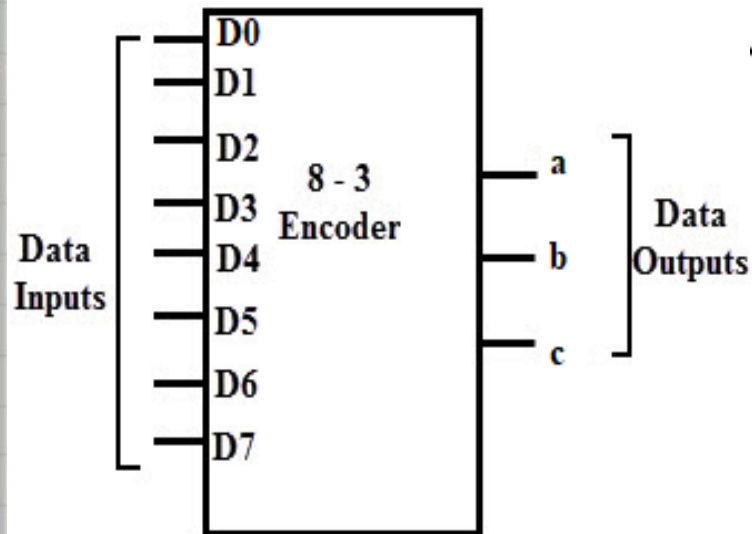
(a) Truth table



(b) Circuit

- 2^N Inputs = N Outputs
- 4 to 2, 8 to 3, 16 to 4

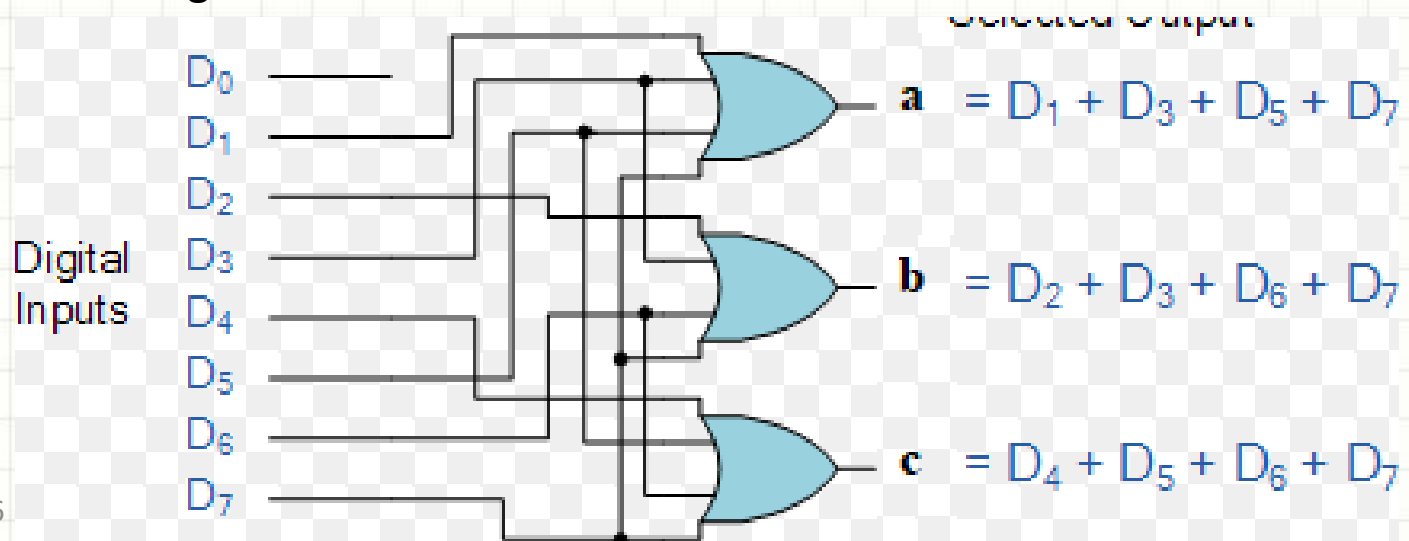
Encoders



Block Diagram

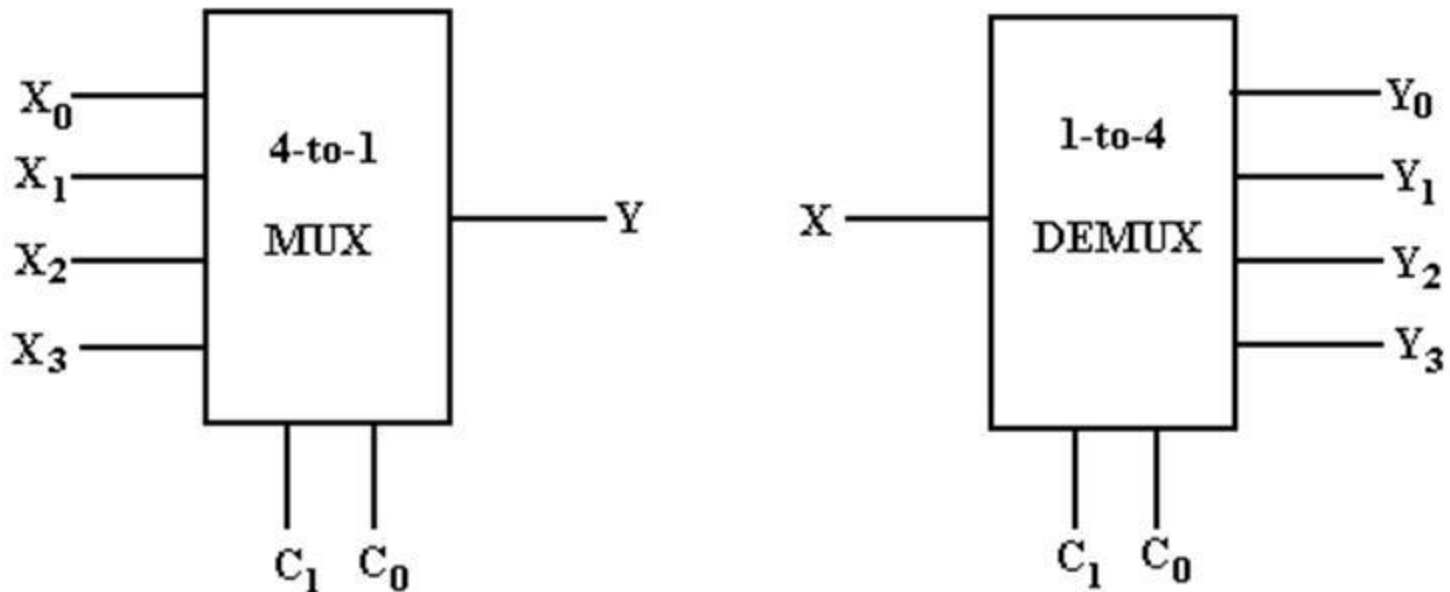
INPUTS								OUTPUTS		
D0	D1	D2	D3	D4	D5	D6	D7	c	b	a
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

Truth Table



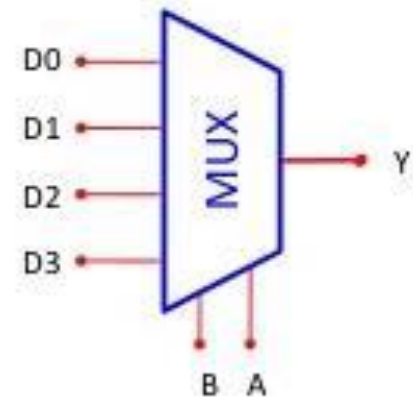
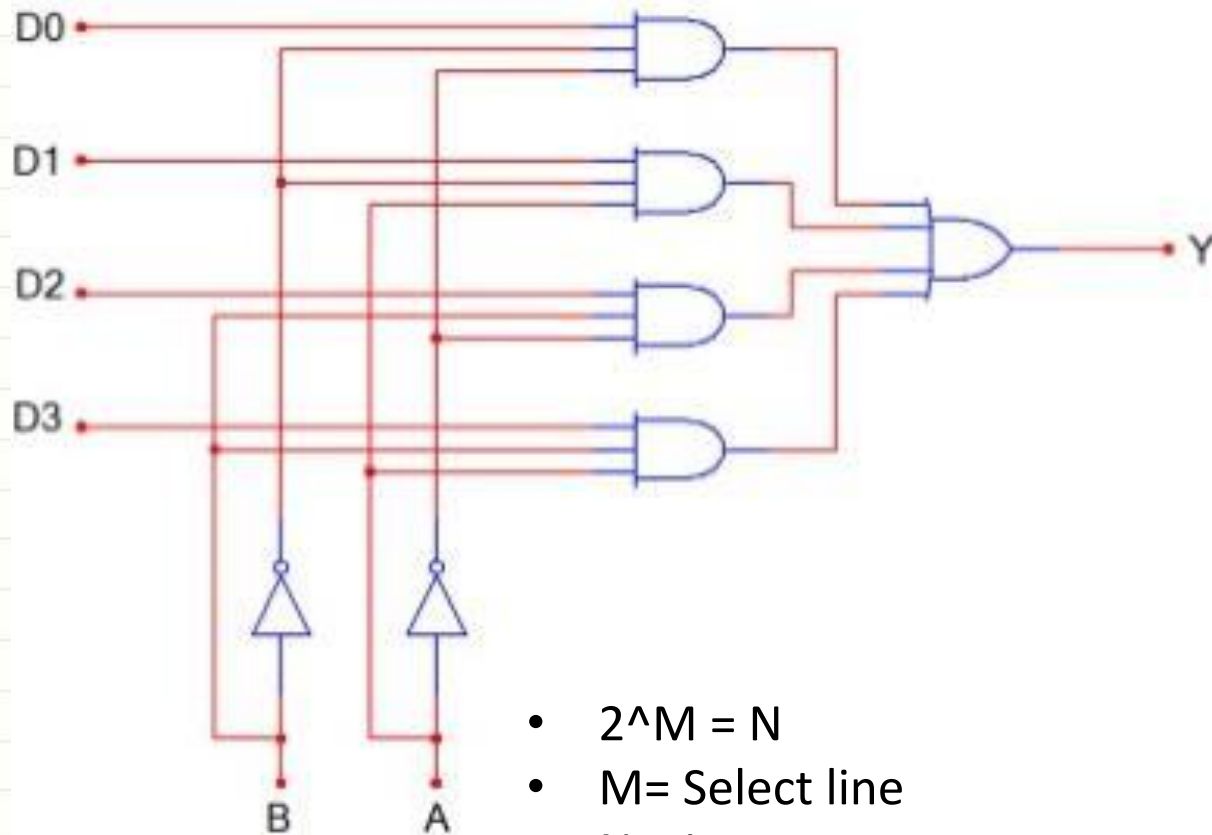
MUX & DEMUX

- Multiplexer is a device that selects one out of several input signals and forwards the single output line.
- Demultiplexer is a device that take single input signal and forwards it to several output line.



Multiplexer

4-to-1 Multiplexer (MUX)

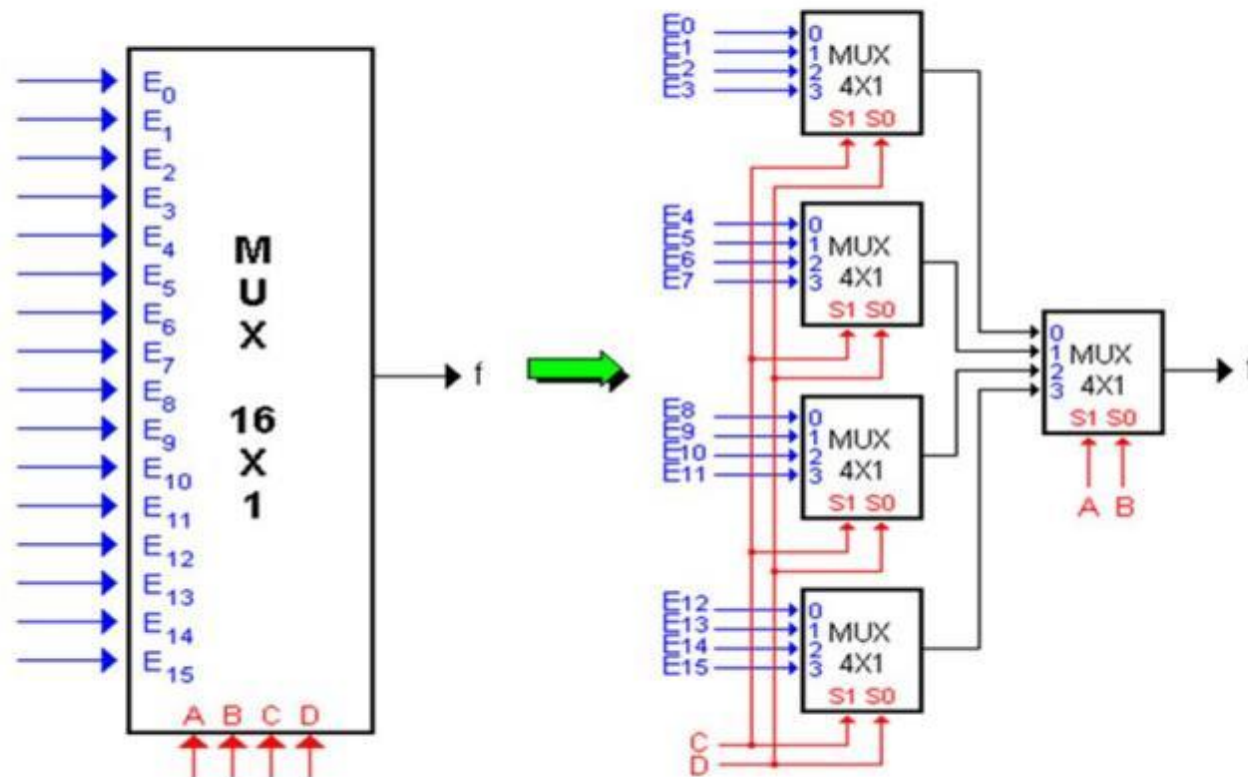


B	A	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

- $2^M = N$
- M = Select line
- N = Input
- 2 to 1, 4 to 1, 8 to 1, 16 to 1

Multiplexer 16 to 1 using 4 to 1

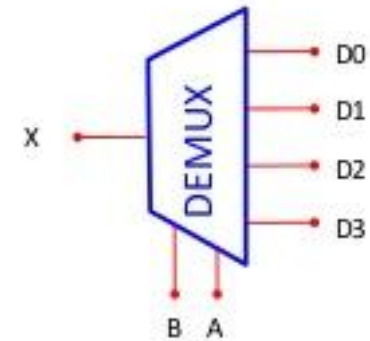
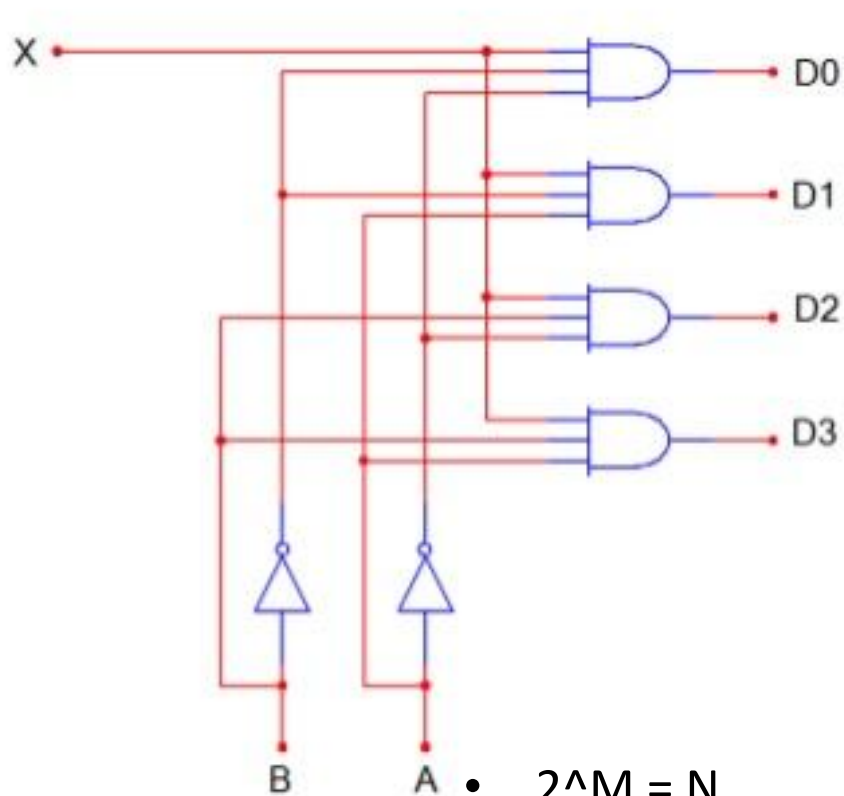
16-to-1 MUX



Practice Example: Draw 8 to 1 using 4 to 1 and 2 to 1.

Demultiplexer.

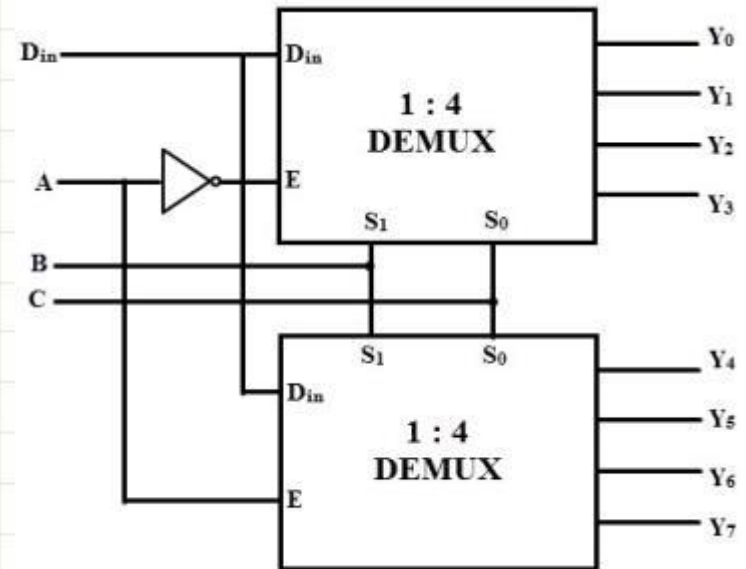
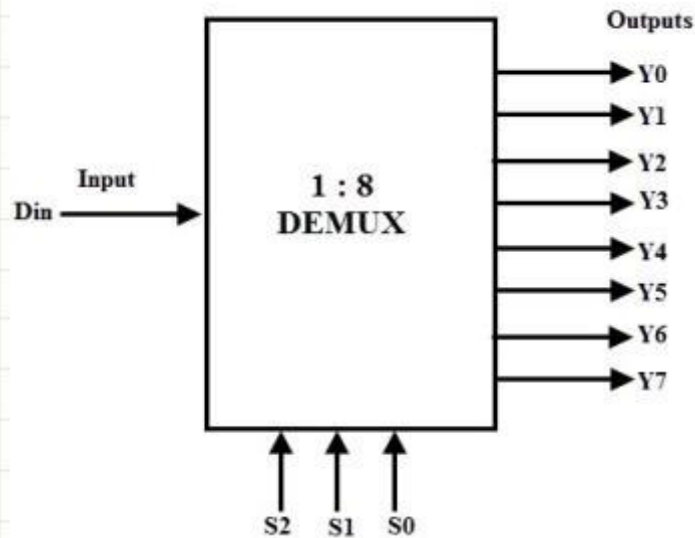
1-to-4 De-Multiplexer (DEMUX)



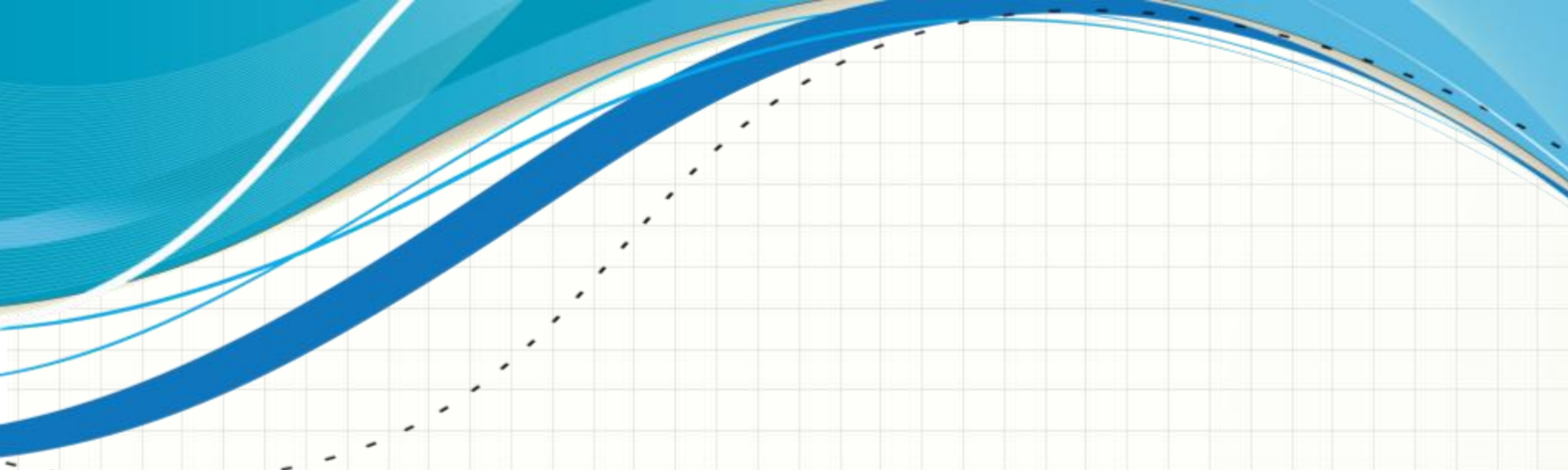
B	A	D0	D1	D2	D3
0	0	X	0	0	0
0	1	0	X	0	0
1	0	0	0	X	0
1	1	0	0	0	X

- $2^M = N$
- M = Select line
- N = Output
- 1 to 2, 1 to 4, 1 to 8, 16 to 1.

Demultiplexer.



Data Input	Select Inputs			Outputs							
D	S_2	S_1	S_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
D	0	0	0	0	0	0	0	0	0	0	D
D	0	0	1	0	0	0	0	0	0	D	0
D	0	1	0	0	0	0	0	0	D	0	0
D	0	1	1	0	0	0	0	D	0	0	0
D	1	0	0	0	0	0	D	0	0	0	0
D	1	0	1	0	0	D	0	0	0	0	0
D	1	1	0	0	D	0	0	0	0	0	0
D	1	1	1	D	0	0	0	0	0	0	0



THANKS